

# 07 - RELACIONAMENTOS E CARDINALIDADES

Banco de Dados • Modelagem Relacional • Projeto SmartCoffee

## Abertura Inspiradora

“Um bom banco de dados não começa nas tabelas. Ele começa entendendo como as informações se relacionam.”

## Apresentação

Na aula anterior, estudamos MER, DER, chaves, tipos de dados e comandos DDL. Agora vamos aprofundar um dos pontos mais importantes da modelagem:

**Como as entidades se relacionam e quantas ocorrências podem participar de cada relacionamento?**

Entender isso evita tabelas mal planejadas, dados duplicados, relacionamentos impossíveis de representar e problemas futuros na aplicação.

## Objetivos da Aula

- identificar relacionamentos entre entidades;
- interpretar cardinalidades **1:1**, **1:N** e **N:N**;
- interpretar cardinalidade mínima e máxima;
- diferenciar participação obrigatória e opcional;
- ler um relacionamento nos dois sentidos;
- transformar relacionamentos N:N em uma estrutura relacional adequada;
- identificar onde uma chave estrangeira deve ser inserida;
- aplicar os conceitos ao banco de dados SmartCoffee.

# 1. Relembrando: o que é um Relacionamento?

Um **relacionamento** representa uma associação existente entre duas ou mais entidades do domínio.

## Entidade

Algo sobre o qual precisamos armazenar informações.

CLIENTE

PEDIDO

PRODUTO

## Relacionamento

Expressa como duas entidades estão conectadas.

**CLIENTE realiza PEDIDO**

**CLIENTE** ————— **realiza** ————— **PEDIDO**

**Dica:** um relacionamento costuma ser identificado por um **verbo**: realiza, possui, pertence, contém, fornece, reserva, utiliza, recebe etc.

## 2. Antes da cardinalidade: faça a pergunta certa

Para descobrir a cardinalidade, leia o relacionamento nos **dois sentidos**.

**Exemplo: CLIENTE e PEDIDO**

**Pergunta 1:** Um CLIENTE pode realizar quantos PEDIDOS?

Resposta: vários pedidos.

**Pergunta 2:** Um PEDIDO pertence a quantos CLIENTES?

Resposta: um cliente.



**CLIENTE 1 : N PEDIDO**

Um cliente pode possuir vários pedidos, mas cada pedido pertence a um único cliente.

**CLIENTE 1 : N PEDIDO**

Um cliente pode possuir vários pedidos, mas cada pedido pertence a um único cliente.

### 3. Cardinalidade Máxima

A cardinalidade máxima informa o **maior número de ocorrências** de uma entidade que pode estar relacionado a uma ocorrência da outra entidade.

| Cardinalidade | Leitura            | Exemplo                                         |
|---------------|--------------------|-------------------------------------------------|
| 1 : 1         | Um para um         | PRODUTO ↔ ESTOQUE, dependendo da regra definida |
| 1 : N         | Um para muitos     | CLIENTE → PEDIDO                                |
| N : N         | Muitos para muitos | FORNECEDOR ↔ PRODUTO                            |

## 4. Relacionamento 1 : 1

No relacionamento **um para um**, uma ocorrência da entidade A se relaciona com no máximo uma ocorrência da entidade B, e vice-versa.



Uma possível regra de negócio seria:

- cada produto possui um único registro de estoque;
- cada registro de estoque pertence a um único produto.

**Atenção:** relacionamentos 1:1 devem ser analisados com cuidado. Em muitos casos, os dados poderiam estar na mesma tabela. O relacionamento separado só faz sentido quando existe uma razão de negócio ou de projeto.

## Onde fica a chave estrangeira?

Em um relacionamento 1:1, a FK pode ficar em uma das tabelas. Para garantir realmente o 1:1, normalmente a FK também precisa ser **UNIQUE**.

```
CREATE TABLE estoque (  
    id_estoque INT PRIMARY KEY AUTO_INCREMENT,  
    id_produto INT NOT NULL UNIQUE,  
    quantidade INT NOT NULL,  
    FOREIGN KEY (id_produto) REFERENCES produto(id_produto)  
);
```

## 5. Relacionamento 1 : N

É o tipo de relacionamento mais comum em bancos relacionais.

**CLIENTE    1    —————    N    PEDIDO**

### Leitura correta

- um CLIENTE pode realizar vários PEDIDOS;
- cada PEDIDO pertence a um único CLIENTE.

**Em um relacionamento 1:N, a chave estrangeira normalmente fica no lado N.**

```
CREATE TABLE pedido (
    id_pedido INT PRIMARY KEY AUTO_INCREMENT,
    data_pedido DATETIME NOT NULL,
    id_cliente INT NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
);
```

**CLIENTE (lado 1) → id\_cliente → PEDIDO (lado N)**

## 6. Relacionamento N : N

Em um relacionamento muitos para muitos, várias ocorrências da primeira entidade podem estar relacionadas a várias ocorrências da segunda.

### Exemplo: FORNECEDOR e PRODUTO

- um fornecedor pode fornecer vários produtos;
- um produto pode ser fornecido por vários fornecedores.

**FORNECEDOR    N    —————    N    PRODUTO**

**Problema:** em um banco relacional, não implementamos diretamente um relacionamento N:N apenas colocando uma FK em uma das tabelas.

### A solução: entidade associativa

O relacionamento N:N é convertido em dois relacionamentos 1:N por meio de uma **entidade associativa**.

**FORNECEDOR    1    —————    N    FORNECEDOR\_PRODUTO    N    —————    1**

| Campo         | Finalidade                            |
|---------------|---------------------------------------|
| id_fornecedor | identifica o fornecedor               |
| id_produto    | identifica o produto                  |
| preco_compra  | preço negociado com aquele fornecedor |
| prazo_entrega | prazo praticado pelo fornecedor       |

```
CREATE TABLE fornecedor_produto (
    id_fornecedor INT NOT NULL,
    id_produto INT NOT NULL,
    preco_compra DECIMAL(10,2),
    prazo_entrega INT,
    PRIMARY KEY (id_fornecedor, id_produto),
    FOREIGN KEY (id_fornecedor) REFERENCES fornecedor(id_fornecedor)
    FOREIGN KEY (id_produto) REFERENCES produto(id_produto)
);
```

## 7. Cardinalidade Mínima e Máxima

Uma modelagem mais completa também informa se a participação é **obrigatória** ou **opcional**.

| Notação | Significado     | Interpretação                                              |
|---------|-----------------|------------------------------------------------------------|
| (0,1)   | zero ou um      | participação opcional, no máximo uma ocorrência            |
| (1,1)   | um e somente um | participação obrigatória e única                           |
| (0,N)   | zero ou muitos  | participação opcional, podendo haver várias ocorrências    |
| (1,N)   | um ou muitos    | participação obrigatória, podendo haver várias ocorrências |

Um cliente pode estar cadastrado sem nunca ter realizado um pedido. Porém, todo pedido deve pertencer a um cliente.

**CLIENTE** (0,N) ——— realiza ——— (1,1) **PEDIDO**

- um cliente pode realizar **zero ou muitos** pedidos;
- um pedido deve estar relacionado a **um e somente um** cliente.

## 8. Participação Obrigatória × Opcional

### Opcional

Cardinalidade mínima = 0.

Exemplo: um cliente pode existir sem ter feito pedido.

(0, N)

### Obrigatória

Cardinalidade mínima = 1.

Exemplo: um pedido precisa pertencer a um cliente.

(1, 1)

**Importante:** a cardinalidade não é definida pelo gosto do desenvolvedor. Ela nasce da **regra de negócio**.

## 9. A regra de negócio muda a cardinalidade

## Cenário A — SmartCoffee exige cadastro

Todo pedido deve estar vinculado a um cliente cadastrado.

CLIENTE (0,N) ————— PEDIDO (1,1)

## Cenário B — SmartCoffee permite venda anônima no balcão

Um pedido pode existir sem cliente identificado.

CLIENTE (0,N) ————— PEDIDO (0,1)

**Conclusão:** antes de desenhar o DER, o analista precisa conversar com o usuário, compreender o processo e registrar as regras.

# 10. Notação Pé de Galinha (Crow's Foot)

Em ferramentas de modelagem, é comum encontrar a notação conhecida como **Crow's Foot**.

| Símbolo conceitual | Leitura         |
|--------------------|-----------------|
| ○                  | zero / opcional |
|                    | um              |
| <                  | muitos          |

○|    = zero ou um  
||    = exatamente um  
○<   = zero ou muitos  
|<    = um ou muitos

Ao usar ferramentas como MySQL Workbench ou brModelo, o importante é aprender a **interpretar o símbolo**, e não apenas decorar o desenho.

## 11. Exemplos no SmartCoffee

| Entidades             | Cardinalidade sugerida | Regra de negócio                                                                    |
|-----------------------|------------------------|-------------------------------------------------------------------------------------|
| CLIENTE × PEDIDO      | 1:N                    | um cliente pode realizar vários pedidos; cada pedido pertence a um cliente          |
| CATEGORIA × PRODUTO   | 1:N                    | uma categoria possui vários produtos; cada produto pertence a uma categoria         |
| PEDIDO × ITEM_PEDIDO  | 1:N                    | um pedido contém vários itens; cada item pertence a um pedido                       |
| PRODUTO × ITEM_PEDIDO | 1:N                    | um produto pode aparecer em vários itens de pedido; cada item referencia um produto |
| FORNECEDOR × PRODUTO  | N:N                    | um fornecedor fornece vários produtos e um produto pode ter vários fornecedores     |
| PRODUTO × ESTOQUE     | 1:1*                   | depende da regra adotada para controle do estoque                                   |
| PEDIDO × PAGAMENTO    | 1:N*                   | permite dividir um pedido em uma ou mais formas/parcelas de pagamento               |

\* Esses relacionamentos devem ser confirmados pelas regras de negócio do projeto.

## 12. Estudo de Caso: PEDIDO, PRODUTO e ITEM\_PEDIDO

Um erro comum é desenhar apenas:

PEDIDO N ————— N PRODUTO

Mas o relacionamento entre pedido e produto possui informações próprias:

- quantidade;
- preço unitário no momento da compra;
- observação do item;
- desconto do item.

Por isso, criamos:

**PEDIDO 1 — N ITEM\_PEDIDO N — 1 PRODUTO**

ITEM\_PEDIDO

-----

id\_item

id\_pedido FK

id\_produto FK

quantidade

preco\_unitario

observacao

**Conceito central:** quando um relacionamento N:N precisa armazenar informações próprias, a entidade associativa deixa claro o significado daquele vínculo.

---

## 13. Como descobrir a cardinalidade passo a passo

1. Identifique as duas entidades.
2. Escreva o relacionamento utilizando um verbo.
3. Pergunte: **“Para uma ocorrência de A, quantas ocorrências de B podem existir?”**
4. Faça a pergunta no sentido contrário.
5. Defina a cardinalidade máxima: 1 ou N.
6. Pergunte se a participação é obrigatória.
7. Defina a cardinalidade mínima: 0 ou 1.
8. Confira se a regra faz sentido no contexto real.

**Fórmula mental:**

“Um(a) \_\_\_\_\_ pode/deve ter quantos(as) \_\_\_\_\_?”

## 14. Erros Comuns

- olhar apenas para um dos lados do relacionamento;
- confundir entidade com relacionamento;
- definir cardinalidade sem consultar a regra de negócio;
- colocar a FK no lado errado de um relacionamento 1:N;
- tentar implementar N:N diretamente;
- criar uma entidade associativa sem compreender por que ela existe;
- ignorar a cardinalidade mínima;
- achar que todo relacionamento precisa ser obrigatório.

## 15. Desafio Rápido — Qual é a Cardinalidade?

Para cada situação, identifique a cardinalidade e justifique.

1. Uma categoria pode possuir vários produtos. Cada produto pertence a apenas uma categoria.

2. Um funcionário pode registrar vários pedidos. Cada pedido é registrado por um funcionário.

3. Um fornecedor comercializa vários produtos, e o mesmo produto pode ser comprado de vários fornecedores.

4. Uma mesa pode existir sem nenhuma reserva futura. Uma reserva deve estar vinculada a uma mesa.

5. Um pedido possui vários itens. Um item de pedido pertence a um único pedido.

## 16. Atividade Orientada — SmartCoffee

Em grupos, analisem o sistema SmartCoffee e definam os relacionamentos entre:

CLIENTE

PEDIDO

PRODUTO

CATEGORIA

FORNECEDOR

ESTOQUE

MESA

RESERVA

PAGAMENTO

FORMA\_PAGAMENTO

### Etapa 1 — Escreva as regras

“Um cliente pode realizar vários pedidos. Cada pedido pertence a um cliente.”

### Etapa 2 — Defina as cardinalidades

| Entidade A | Relacionamento | Entidade B | Cardinalidade | Obrigatório? |
|------------|----------------|------------|---------------|--------------|
| CLIENTE    | realiza        | PEDIDO     |               |              |
| CATEGORIA  | possui         | PRODUTO    |               |              |
| FORNECEDOR | fornece        | PRODUTO    |               |              |
| MESA       | recebe         | RESERVA    |               |              |
| PEDIDO     | recebe         | PAGAMENTO  |               |              |

## Etapa 3 — Resolva pelo menos um N:N

- nome da tabela;
- chaves estrangeiras;
- possíveis atributos do relacionamento;
- chave primária.

## Etapa 4 — Desenhe o DER

Represente pelo menos **6 entidades**, indicando PK, FK, relacionamentos, cardinalidade mínima e cardinalidade máxima.

---

## 17. Desafio de Análise

### Situação:

Um aluno criou a tabela PEDIDO contendo:

`id_pedido, data, id_cliente, produto1, produto2, produto3, quantidade1, quantidade2, quantidade3`

1. Qual é o problema dessa estrutura?
  2. Qual relacionamento existe entre PEDIDO e PRODUTO?
  3. Qual entidade deveria ser criada para resolver o problema?
  4. Quais FKs essa entidade possuiria?
  5. Que outros atributos poderiam existir nessa entidade?
- 

## 18. Conectando Cardinalidade com DDL

O DER não é apenas um desenho. As decisões de relacionamento aparecem diretamente na criação das tabelas.

```

CREATE TABLE cliente (
    id_cliente INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL
);

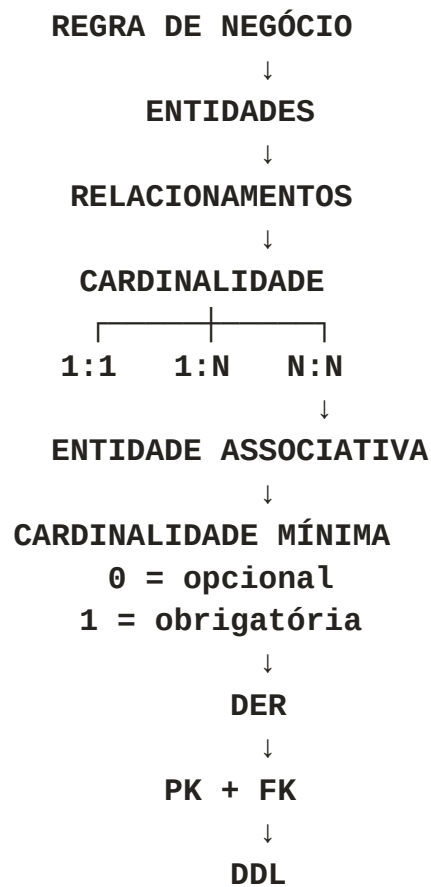
CREATE TABLE pedido (
    id_pedido INT PRIMARY KEY AUTO_INCREMENT,
    data_pedido DATETIME NOT NULL,
    id_cliente INT NOT NULL,

    CONSTRAINT fk_pedido_cliente
        FOREIGN KEY (id_cliente)
        REFERENCES cliente(id_cliente)
);

```

**CLIENTE 1 : N PEDIDO → FK id\_cliente dentro de PEDIDO**

## 19. Mapa Mental



# Encerramento

**Antes de criar uma Foreign Key, pergunte: “qual regra de negócio essa chave representa?”**

Na próxima etapa, o modelo poderá evoluir para a implementação dos relacionamentos no MySQL, criação de constraints e manipulação dos dados relacionados.